



Unidad 5

Patrones creacionales: quién hace el new

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026



**Un patrón de diseño es una
solución probada a un problema
que se repite.**

**Se gana un lenguaje en común, por ejemplo: Observer "el
modelo es observable".**



Tres grupos

- **Creacionales:** cómo se crean los objetos. Hoy.
- **Estructurales:** cómo se combinan clases y objetos en estructuras más grandes. El miércoles.
- **De comportamiento:** cómo se reparten el trabajo y se comunican. La semana que viene.
- La referencia para leerlos: refactoring.guru/es/design-patterns, gratis y en castellano, con diagramas y código Java.

Gamma, Helm, Johnson y Vlissides, 1994. El libro de "Gang of Four".

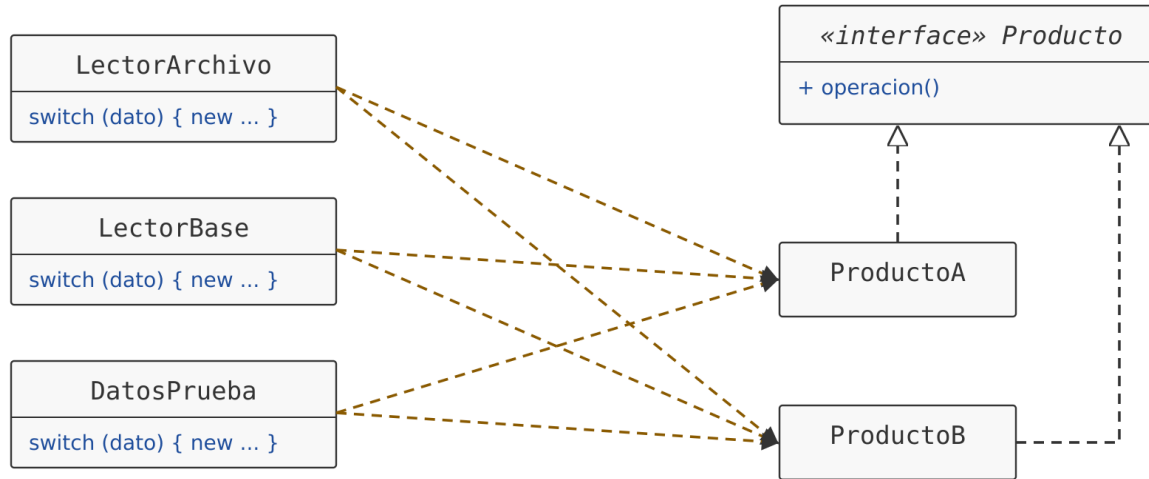


El único lugar que necesita la clase concreta

- Casi todo el código trabaja con abstracciones: un **Pago**, una lista de **Producto**.
- El **new** no puede. Para crear un objeto hay que nombrar su clase.
- El problema no es el **new**, es el **new** repetido: cada lugar que decide queda atado a las clases concretas.
- El polimorfismo no ayuda acá: antes del **new** no hay ningún objeto a quien preguntarle.

Los patrones creacionales reducen ese lugar a uno.

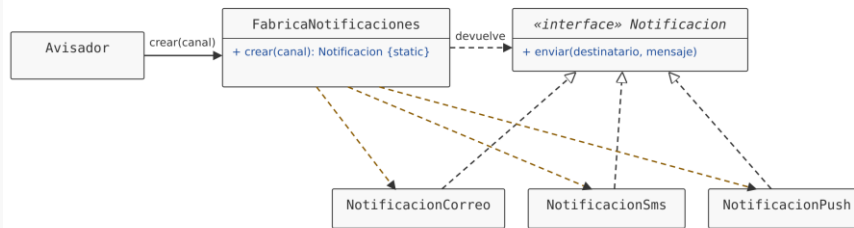
Factory: el problema



Tres lugares deciden la clase. Seis `new` para dos productos, y un tipo nuevo obliga a abrir los tres archivos.

Factory: un método de creación

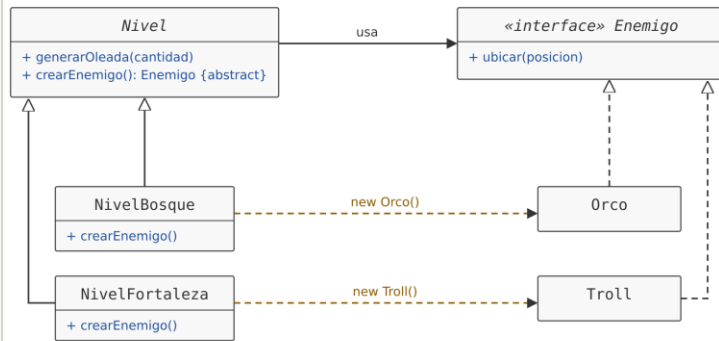
```
public class FabricaNotificaciones {  
  
    public static Notificacion crear(String canal) {  
        switch (canal) {  
            case "correo": return new NotificacionCorreo();  
            case "sms":     return new NotificacionSms();  
            case "push":    return new NotificacionPush();  
            default:  
                throw new IllegalArgumentException(  
                    "Canal desconocido: " + canal);  
        }  
    }  
}
```



El switch existe, pero existe una vez. El resto del programa trabaja con **Notificacion**.

Factory Method: la subclase hace el new

```
public abstract class Nivel {  
    public void generarOleada(int cantidad) {  
        for (int i = 0; i < cantidad; i++) {  
            Enemigo e = this.crearEnemigo(); // factory method  
            e.ubicar(this.posicionLibre());  
            this.enemigos.add(e);  
        }  
    }  
    protected abstract Enemigo crearEnemigo();  
}  
  
public class NivelBosque extends Nivel {  
    protected Enemigo crearEnemigo() { return new Orco(); }  
}
```



La clase base hace el proceso y la subclase hace el new. Un nivel nuevo es una clase con un método.

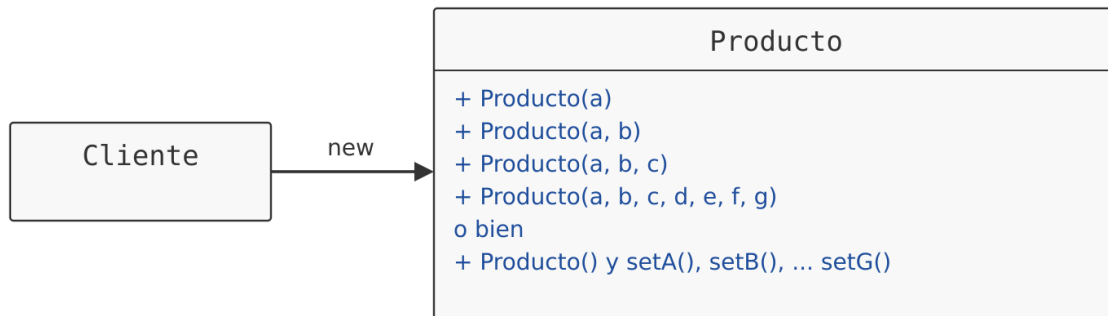


Factory o Factory Method: quién elige

- **Factory:** el que llama pasa un dato, y el **switch** decide. Sirve cuando la clase depende de un texto, una opción, una columna.
- **Factory Method:** el que llama ya eligió al crear un **NivelBosque**. No hay dato ni **switch**: cada subclase es la decisión.
- Un tipo nuevo: un **case** en el primero, una clase en el segundo. Open/Closed en el segundo.
- Ninguno hace magia. Nombrar la clase concreta es inevitable, una vez.

Si la decisión viene de un dato, **Factory**. Si viene de la variante, **Factory Method**.

Builder: el problema



`new Producto("x", null, 0, "", true, null, 0)`

`new Producto(); p.setA("x"); ... p.setG(0);`

Un constructor por combinación, o uno de siete parámetros donde no se sabe qué es cada valor, o un objeto incompleto entre el `new` y el último `set`.

Builder: lo obligatorio en el constructor, lo opcional con nombre

```
public static class Builder {  
    private final int habitacion;           // obligatorio  
    private final int noches;              // obligatorio  
    private boolean desayuno = false;      // opcional, con valor por defecto  
    private boolean cochera = false;  
  
    public Builder(int habitacion, int noches) { ... }  
  
    public Builder conDesayuno() { this.desayuno = true; return this; }  
    public Builder conCochera() { this.cochera = true; return this; }  
  
    public Reserva construir() { return new Reserva(this); } // valida todo  
}
```

Cada método devuelve `this` para encadenar. El constructor de `Reserva` es privado y valida antes de asignar.

Builder: se lee como se dice

```
Reserva fin = new Reserva.Builder(204, 3)
    .conDesayuno()
    .conCochera()
    .construir();
```

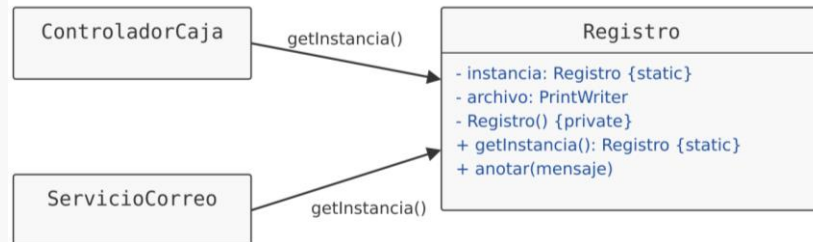
```
// Sin Builder:
// new Reserva(204, 3, true, true, false)
// ¿cuál es la cuna? ¿cuál la cochera?
```



El objeto nace completo y validado, o no nace. Todos sus atributos pueden ser **final**.

Singleton: una instancia, garantizada por la clase

```
public class Registro {  
    private static Registro instancia;    // la unica  
    private final PrintWriter archivo;  
  
    private Registro() throws IOException { // nadie hace new  
        archivo = new PrintWriter(  
            new FileWriter("aplicacion.log", true));  
    }  
  
    public static Registro getInstancia() throws IOException {  
        if (instancia == null) instancia = new Registro();  
        return instancia;  
    }  
}
```



Constructor privado, atributo estático, `getInstancia()`. Todas las clases escriben en el mismo archivo.



Singleton, con honestidad

- Garantiza dos cosas: instancia única, que es lo que se quería, y acceso global, que no.
- Un **getInstancia()** adentro de una clase la ata a ese objeto y no se puede reemplazar, ni siquiera en una prueba.
- **getInstancia()** se llama en **main** y el objeto viaja por constructor.
- Solo para lo que no puede existir dos veces: una conexión, un archivo abierto, la placa de sonido. "Una sola partida" es una porque el programa creó una, no es Singleton.

Que haya uno no es lo mismo que no pueda haber dos.



En la biblioteca

- **Factory:** `Integer.valueOf(3)`, `List.of("a", "b")`, `LocalDate.of(2026, 9, 28)`. Un método estático decide qué objeto devolver.
- **Builder:** `StringBuilder`, con `append()` devolviendo `this` y `toString()` como `construir()`.
- **Singleton:** `Runtime.getRuntime()`, `Toolkit.getDefaultToolkit()`. Constructor privado y un método estático que siempre devuelve el mismo objeto.

Tres maneras que tiene la biblioteca de no hacer escribir `new` a quien la usa.



La actividad de hoy

- Un juego de oleadas en consola, con sonido de verdad (`javax.sound.sampled`, sin bibliotecas), y tres problemas: el gestor de audio se abre tres veces, el héroe tiene un constructor de ocho parámetros, y el nivel decide el enemigo con un **switch** sobre un texto.
- **Parte 1:** ejecutar y escuchar, contar los `new GestorAudio()`, adivinar cuál argumento es la montura, y ver cómo falla el pantano sin avisar.
- **Parte 2:** Singleton para el audio (y se inyecta), Builder para el héroe, Factory Method para el nivel.
- **Parte 3:** `NivelPantano` con arqueros, sin tocar `Nivel`.